Priority-Based Greedy Execution: Applying Job Scheduling to Trading

Problem Statement

Your current greedy algorithm is **order-dependent**: if you have 10 signals but only 2.0x leverage capacity, whichever signals come first in dictionary iteration get executed, and later signals get blocked—regardless of their confidence or importance.

```
Dictionary iteration order (arbitrary):
Signal A: +0.15 confidence 0.90 ✓ FILLED
Signal B: +0.12 confidence 0.88 ✓ FILLED
Signal C: +0.18 confidence 0.95 ✗ BLOCKED (leverage exhausted)
Signal D: +0.10 confidence 0.70 ✗ BLOCKED
```

Better: Execute by priority (confidence × signal strength):

```
Sorted by (confidence × magnitude):
Signal C: +0.18 × 0.95 = 0.171 ✓ FILLED
Signal A: +0.15 × 0.90 = 0.135 ✓ FILLED
Signal B: +0.12 × 0.88 = 0.106 ✗ BLOCKED (leverage exhausted)
Signal D: +0.10 × 0.70 = 0.070 ✗ BLOCKED
```

Solution: Scheduling with Capacity Constraints

Map your problem to **classic job scheduling**:

Concept	Job Scheduling	Trading Execution
Jobs	Tasks to schedule	Signals/symbols
Profit	Job profit value	Signal strength + confidence
Deadline	Time limit	Leverage availability
Capacity	CPU time	Max gross leverage (e.g., 200%)
Algorithm	Sort by profit/deadline ratio, greedily fit	Sort by priority score, execute while budget available

---

Algorithm Implementation

Step 1: Compute Priority Score for Each Signal

```
def compute_signal_priority(signal, ml_confidence):
    """
    Priority = (signal_strength × ml_confidence) × urgency_factor

    Higher values = execute first
    """
    base_priority = abs(signal.target_pct) * (ml_confidence or 1.0)

    # Optional: urgency factor (e.g., higher for recently triggered signals)
    # urgency = 1.0 + (time_since_signal / 10) if within last 10 bars else 1.0

    return base_priority
```

Step 2: Sort Signals by Priority

```
def sort_signals_by_priority(signals, ml_preds):
    """
    Sort signals in descending order of priority.
    High-priority signals execute first.
    """
    scored_signals = []

    for sym, sig in signals.items():
```

```
        ml_confidence = None
        if sym in ml_preds:
            p = ml_preds[sym].loc[:ts].tail(1)
            if len(p) == 1:
                ml_confidence = float(p.iloc[0])

        priority = compute_signal_priority(sig, ml_confidence)
        scored_signals.append({
            'symbol': sym,
            'signal': sig,
            'ml_confidence': ml_confidence,
            'priority': priority,
        })

    # Sort by priority descending (highest priority first)
    return sorted(scored_signals, key=lambda x: x['priority'], reverse=True)
```

Step 3: Execute with Leverage Tracking

```
def execute_with_priority_scheduling(
    signals,
    ml_preds,
    pf,
    prices,
    equity,
    limits,
    broker,
    ts
):
    """
    Execute signals in priority order, respecting leverage constraints.

    This is the "Scheduling with Deadlines" greedy algorithm applied to trading.
    """

    # Sort signals by priority
    sorted_signals = sort_signals_by_priority(signals, ml_preds)

    executed = []
    blocked = []
    leverage_used = 0.0

    for scored_sig in sorted_signals:
        sym = scored_sig['symbol']
        sig = scored_sig['signal']
        ml_confidence = scored_sig['ml_confidence']
        priority = scored_sig['priority']

        px = prices.get(sym)
        if px is None:
            continue

        # Compute target position
        target_pct = clamp_target(sym, sig.target_pct, equity, px, limits)

        # Apply ML scaling
        if ml_confidence:
            if ml_confidence >= 0.5:  # Only scale if reasonably confident
                scale = max(min((ml_confidence - 0.5) * 2.0, 1.0), -1.0)
                target_pct = target_pct * scale

        # Compute delta
        current_qty = pf.position_qty(sym)
        target_qty = target_pct_to_qty(target_pct, equity, px)
        delta = target_qty - current_qty

        # Skip tiny rebalances
        if abs(delta) * px < 10.0:
            blocked.append({
                'symbol': sym,
                'reason': 'filter',
                'priority': priority,
                'delta_notional': delta * px,
            })
            continue

        # Check if this execution would exceed leverage
        delta_leverage = (abs(delta) * px) / (equity + 1e-8)
        new_leverage = leverage_used + delta_leverage
```

```
        if new_leverage > limits.max_gross_leverage:
            # Cannot execute: at leverage limit
            blocked.append({
                'symbol': sym,
                'reason': 'leverage',
                'priority': priority,
                'delta_notional': delta * px,
                'leverage_would_be': new_leverage,
            })
            continue

        # CAN execute: update leverage and place order
        leverage_used = new_leverage
        fill = broker.place_order(Order(sym, qty=delta), ts=ts)
        pf.update_from_fill(fill)

        executed.append({
            'symbol': sym,
            'priority': priority,
            'ml_confidence': ml_confidence,
            'target_pct': target_pct,
            'delta_notional': delta * px,
            'leverage_contribution': delta_leverage,
            'status': 'FILLED',
        })

    return {
        'executed': executed,
        'blocked': blocked,
        'total_leverage': leverage_used,
        'leverage_utilization': leverage_used / limits.max_gross_leverage,
    }
```

Comparison: Sequential vs. Priority-Based

Example: 5 Signals, 2.0x Max Leverage

Current (Sequential/Random Order):

```
Signal Order   | Target | Conf   | Notional | Cumul Leverage | Status
---------------|--------|--------|----------|----------------|----------
BTC (1st)      | +15%   | 0.90   | $15M     | 0.75 (150%)    | ✓ FILLED
ETH (2nd)      | +12%   | 0.88   | $12M     | 1.35 (270%)    | ✗ BLOCKED ← leverage
SOL (3rd)      | +18%   | 0.95   | $18M     | — (would be 1.65) | ✗ BLOCKED
ADA (4th)      | +10%   | 0.70   | $10M     | —              | ✗ BLOCKED
DOT (5th)      | +8%    | 0.65   | $8M      | —              | ✗ BLOCKED

Total Executed: $15M + $12M = $27M (some capital wasted on low-confidence ETH)
```

Priority-Based Scheduling:

```
Sorted by (|Target| × Conf):
Priority | Symbol | Target | Conf  | Score  | Notional | Cumul Leverage | Status
---------|--------|--------|-------|--------|----------|----------------|----------
1 (HIGH) | SOL    | +18%   | 0.95  | 0.171  | $18M     | 0.90 (180%)    | ✓ FILLED
2        | BTC    | +15%   | 0.90  | 0.135  | $15M     | 1.65 (330%)    | ✗ BLOCKED ← leverage
3        | ETH    | +12%   | 0.88  | 0.106  | $12M     | —              | ✗ BLOCKED
4        | ADA    | +10%   | 0.70  | 0.070  | $10M     | —              | ✗ BLOCKED
5 (LOW)  | DOT    | +8%    | 0.65  | 0.052  | $8M      | —              | ✗ BLOCKED

Total Executed: $18M (highest-confidence signal fills!)
Capital efficiency: 90% vs 67% in sequential approach
```

Key Win: SOL (highest confidence) executes instead of ETH (lowest confidence).

UI/UX Changes for Priority-Based Execution

1. Execution Queue Shows Priority

```
<ExecutionQueueModal>
  Priority | Symbol | Target | Conf  | Score  | Status
  ─────────┼────────┼────────┼───────┼────────┼──────────
  1        | SOL    | +18%   | 0.95  | 0.171  | ✓ FILLED
  2        | BTC    | +15%   | 0.90  | 0.135  | □ WAITING
  3        | ETH    | +12%   | 0.88  | 0.106  | □ BLOCKED

  "High-priority signals execute first.
   SOL (strongest signal) filled.
   BTC waiting for leverage headroom."
</ExecutionQueueModal>
```

2. Watchlist Sorts by Priority

Add "Execution Order" column:

```
Symbol | Signal | Conf | Priority Score | Order | Status
───────┼────────┼──────┼────────────────┼───────┼──────────
SOL    | +18%   | 0.95 | 0.171          | 1st   | ✓ FILLED
BTC    | +15%   | 0.90 | 0.135          | 2nd   | □ QUEUED
ETH    | +12%   | 0.88 | 0.106          | 3rd   | □ BLOCKED
ADA    | +10%   | 0.70 | 0.070          | 4th   | □ BLOCKED
```

3. Explainer Tooltip

Hovering over status shows:

```
"This signal is blocked because previous high-priority
 signals (SOL, BTC) consumed the 2.0x leverage limit.

 To execute this signal, either:
 • Close a lower-priority position (ADA, DOT)
 • Wait for SOL/BTC to rebalance down
 • Reduce the target size"
```

Code Integration: Updated Backtest Loop

```
# In backtest CLI (line ~1403)

# BEFORE: naive sequential execution
# for sym, sig in sigs.items():
#     ... execute ...

# AFTER: priority-scheduled execution
result = execute_with_priority_scheduling(
    signals=sigs,
    ml_preds=ml_preds,
    pf=pf,
    prices=prices,
    equity=float(row["equity"]),
    limits=limits,
    broker=broker,
    ts=ts
)

# Track execution history
signal_history.append({
    "timestamp": ts,
    "executed": result['executed'],
    "blocked": result['blocked'],
    "total_leverage": result['total_leverage'],
    "leverage_utilization": result['leverage_utilization'],
})

# Update portfolio for each filled order
for exec in result['executed']:
    # Already done in execute_with_priority_scheduling
    pass
```

Metrics: Before & After

Add these to your `metrics.json`:

```
{
  "execution_approach": "priority_scheduling",
  "execution_efficiency": {
    "total_signals": 10,
    "signals_executed": 3,
    "signals_blocked": 7,
    "avg_priority_of_executed": 0.158,
    "avg_priority_of_blocked": 0.065,
    "priority_ratio": 2.43,
    "comment": "Executed signals were 2.43x higher priority than blocked ones"
  },
  "leverage_metrics": {
    "max_leverage_used": 0.9,
    "avg_leverage_used": 0.65,
    "times_at_max": 5,
    "leverage_efficiency": "High - executing only high-priority signals"
  }
}
```

Implementation Checklist

Backend

- Implement `compute_signal_priority()`
- Implement `sort_signals_by_priority()`
- Implement `execute_with_priority_scheduling()`
- Add execution order tracking to signal_history
- Update metrics export

Frontend

- Add "Priority Score" column to watchlist
- Add "Execution Order" column
- Update ExecutionQueueModal to show sorted order
- Add explainer tooltip for blocked signals
- Show "why blocked" reasoning

Testing

- Backtest with random signal order → verify same results as priority-based
- Backtest with high/low confidence mix → verify high-conf fills first
- Compare total return: priority vs sequential

Advanced: Multiple Constraint Types

If you later want to handle multiple constraints (leverage, position limit, daily loss):

```
def can_execute_signal(signal, constraints):
    """
    Check if signal can execute given ALL constraints.
    Returns (can_execute, blocking_constraint)
    """
    checks = [
        ('leverage', signal.leverage_impact < constraints.leverage_remaining),
        ('max_position', signal.target_pct < constraints.max_position_pct),
        ('daily_loss', constraints.daily_loss < constraints.max_daily_loss),
        ('notional_min', abs(signal.delta_notional) > 10.0),
    ]

    for constraint_type, passes in checks:
        if not passes:
```

```
            return False, constraint_type

    return True, None
```

Then in scheduling loop:

```
can_exec, blocking = can_execute_signal(sig, constraints)
if not can_exec:
    blocked.append({
        'symbol': sym,
        'reason': blocking,
        'priority': priority,
    })
    continue
```

Summary

Your intuition was spot-on: **apply classical greedy algorithm patterns** to the execution problem.

Before: Random order → low fairness **After**: Priority scheduling → highest-confidence signals always fill first

This is a **small code change** (~50 lines) but **big UX win**:

- Users understand execution order
- High-confidence signals preferentially execute
- Blocked signals show clear "why" explanation
- Leverage is used more efficiently

Start with: Implement `execute_with_priority_scheduling()` + update watchlist to show priority order. That's the core value.